

Assessment and Feedback Handbook

Question writing, practical tasks, rubrics, feedback, and assessment review.

Reference material for lesson planning and course-content development.

Contents

1. What assessment is for
2. Writing multiple-choice questions
3. Short-answer and explanation questions
4. Prediction and tracing questions
5. Debugging and error-analysis questions
6. Practical and build tasks
7. Difficulty without trick questions
8. Answer keys and explanations
9. Rubrics for small projects and artifacts
10. Feedback that helps revision
11. Reviewing an assessment before release

1. What assessment is for

Assessment should provide evidence about what a learner can do. A quiz is not automatically useful because it has many questions. The questions need to connect to the lesson objectives and require the kind of thinking the objective names.

Formative assessment happens during learning and helps decide what to explain or practice next. Summative assessment is used to judge achievement at the end of a unit or course. A short lesson pack usually needs a small formative or end-of-lesson check, not a full exam.

Alignment

If the objective is to debug a loop, at least one assessment item should involve debugging. Five questions asking for definitions of for-loop and while-loop do not test the same skill.

2. Writing multiple-choice questions

A multiple-choice item should have one best answer and distractors that a partially prepared learner might reasonably choose. Weak distractors make the question a guessing exercise.

Good distractors

- Reflect a common misconception.
- Use the wrong result from a common calculation error.
- Describe a related concept that does not answer this exact question.
- Represent an approach that could work in another situation but is weaker here.

Avoid making the correct option consistently longer, more detailed, or more carefully qualified than the others. Avoid nonsense options added only to reach four choices.

Stems

The stem should contain the problem. Do not force learners to reread four long options to understand what is being asked. Scenario-based stems can be useful when the scenario is necessary to test application rather than recall.

3. Short-answer and explanation questions

Short-answer questions are useful when the learner needs to explain a distinction, interpret a result, or justify a choice. The marking guide should identify the important points rather than require one exact sentence.

Examples

- Explain why the while-loop stops.
- Why is the median a better summary than the mean for this dataset?
- Which force pair is a Newton's third-law pair, and why?
- What is wrong with the claim in this project update?

Keep the expected answer proportional to the available time. If a two-mark item expects a full essay, the task is poorly calibrated.

4. Prediction and tracing questions

Prediction questions are especially useful in programming, mathematics, and science because they expose whether the learner can follow a process. Ask for the next state, final output, or likely result before execution.

Programming

Show a short loop and ask for output or the value of a variable after each iteration. Keep the code short enough to trace.

Mathematics

Show the first two algebra steps and ask which next step preserves equality. Or give a graph and ask how y changes when x increases.

Science

Describe a change in force, mass, temperature, or concentration and ask for the expected direction of effect before numerical calculation.

5. Debugging and error-analysis questions

Error-analysis questions ask learners to identify why a believable solution is wrong. They are valuable because they test more than recognition of a correct finished answer.

Structure

- Provide a short incorrect solution or output.
- Ask for the first important error.
- Ask why it is wrong.
- Optionally ask for a corrected step.

The error should be realistic. A Python loop that forgets to update its condition variable is realistic. A loop that contains random words instead of code is not useful.

6. Practical and build tasks

A practical task asks learners to produce something: a small program, a graph, a balanced equation, an email, or a set of meeting notes. The marking criteria should focus on required behavior and quality rather than one exact implementation.

Programming example

Write a function that takes a list of prices and returns the total. The marking guide can check correct iteration, accumulation, return value, and handling of an empty list if that case was taught.

Communication example

Rewrite a project update so that progress, blocker, and next step are clear. The marking guide can check clarity, completeness, tone, and actionable next step.

For beginner tasks, syntax mistakes may be less important than whether the structure shows the intended reasoning, depending on the purpose of the assessment.

7. Difficulty without trick questions

A harder question should require more reasoning with material that was taught. It should not become difficult because of obscure vocabulary, hidden assumptions, or content from outside the lesson.

Ways to increase difficulty

- Combine two ideas that were both taught.
- Use a less familiar context.
- Remove one hint.
- Ask the learner to choose between two plausible methods.
- Ask for diagnosis rather than definition.
- Require interpretation of intermediate output.

A strong question may have two options that seem plausible at first, with one being better because of a condition in the scenario. The distinction should be defensible from the lesson material.

8. Answer keys and explanations

An answer key should be useful to the person reviewing the work. For objective questions, include the answer and a short reason when a misconception is likely. For open tasks, list the expected features or steps.

Good explanation

For a range question, do not only say 'B'. Explain that the stop value is excluded. For a graph question, explain what the slope represents in the given units. For a science question, state which force acts on which object.

Avoid explanations that introduce new theory that was not needed to answer the question. The answer key is not the place to add an extra lesson.

9. Rubrics for small projects and artifacts

A rubric is useful when the output can vary. Keep the number of categories small enough that a reviewer can use it consistently. Four or five categories are often enough for a short lesson artifact.

Category	What to look for
Accuracy	Content is correct and conditions are preserved
Alignment	Output matches the requested topic and level
Clarity	Explanations and instructions are easy to follow
Practice	Tasks reflect what was taught and vary appropriately
Completeness	Required sections are present

Descriptions should distinguish levels clearly. Avoid vague labels such as 'excellent' without saying what excellent work contains.

10. Feedback that helps revision

Feedback should identify what needs to change and why. 'Wrong' is less useful than 'the loop never changes count, so the condition remains true'. Specific feedback gives the learner a next action.

Useful feedback pattern

- Name the part that worked.
- Identify the specific issue.
- Explain why it matters.
- Suggest the next change or question to check.

Do not rewrite the entire answer when the goal is for the learner to revise it. Give enough direction to support the next attempt.

11. Reviewing an assessment before release

Read every item after the lesson is complete. Check whether the lesson actually taught the knowledge required to answer it. Then review the options, answer key, and wording.

Checklist

- Is there one best answer?
- Are distractors plausible?
- Is the correct answer not consistently longer?
- Does the item test the intended objective?
- Is the difficulty coming from reasoning rather than missing information?
- Are calculations and answer keys correct?
- Are instructions clear about what the learner should produce?
- Does the set include more than definition recall?

A short assessment can still be strong. Ten carefully chosen questions are often more informative than thirty repetitive ones.